

RGUKT Connect

Technical and Product Documentation

A secure, dedicated professional networking platform bridging alumni and students of Rajiv Gandhi University of Knowledge Technologies.

Project Context: Professional Peer Networking, Referrals, and Mentorship
Target Audience: Software Engineers, Recruiters, Collaborators, and Alumni

Executive Summary

RGUKT Connect is an enterprise-grade professional networking and mentorship platform designed exclusively for the community of the Rajiv Gandhi University of Knowledge Technologies (RGUKT). Built to establish a secure, verified digital bridge between current university students and alumni working in the industry, the platform facilitates direct career guidance, verified job referrals, and real-time collaboration.

Key Capabilities

- **Secure Verification:** Restricts enrollment to authorized university members through a dual-step identity verification requiring a valid '@rgukt.ac.in' domain and individual student ID numbers validated via automated One-Time Passwords (OTP).
- **Verified Professional Portfolios:** Offers comprehensive profiles containing detailed academic histories, verified work experiences, project source linkages (GitHub/LinkedIn), and academic branches.
- **Mentorship and Alumni Directory:** Provides a searchable directory of the RGUKT network, allowing students to easily discover and reach out to seniors who possess active careers in specific organizations.
- **Direct Referrals & Job Board:** Enables verified alumni and administrators to publish career opportunities and offer direct referrals, skipping the generic application process.
- **Real-Time Messaging:** Employs a STOMP-based WebSocket communication layer to enable instant, persistent messaging between connected members.

Technical Overview

RGUKT Connect features a modern split-architecture. The backend is powered by Java 21, Spring Boot 3/4, Spring Security, Hibernate ORM, and a PostgreSQL database. It offloads binary assets (such as profile pictures and post media) directly to Amazon S3. The frontend is built as a highly responsive single-page application using React 19, Vite, Tailwind CSS 4.0, and Axios. Real-time notifications and message deliveries are bridged using a STOMP messaging broker over WebSockets. Deployment is optimized using containerized Docker environments managed by a Jenkins automation pipeline and orchestrated inside a Kubernetes cluster.

Contents

Executive Summary	1
1 Introduction	8
1.1 Project Motivation	8
1.2 System Objectives	8
1.3 Document Scope	9
2 Problem Statement	10
2.1 Analysis of Current Communication Deficits	10
2.2 The Verification Gap and Trust Deficit	10
2.3 The Job Referral Bottleneck	11
2.4 The Proposed Solution: RGUKT Connect	11
3 Product Overview	12
3.1 Target User Roles	12
3.2 Platform Boundaries	12
3.3 User Journey & Flow	12
4 Features	14
4.1 Identity Verification & OTP Lifecycle	14
4.2 Alumni Directory & Connection Management	14
4.3 Real-Time STOMP Messaging	15
4.4 Opportunities Board & Job Referrals	15
4.5 Community Feed & Media Integration	15
4.6 Profile Customization	15
5 User Roles and Permissions	16
5.1 Role Profiles	16
5.2 Permission Matrix	16
5.3 Security Policy Enforcements	17
6 System Architecture	18
6.1 System Architecture Overview	18
6.2 Component Communication Protocols	19
7 Technology Stack	20
7.1 Tech Stack Matrix	20
7.2 Technical Rationale for Primary Choices	21

8	Application Workflows	23
8.1	Registration & OTP Verification Workflow	23
8.2	Authentication & JWT Lifecycle	23
8.3	Job Posting & Referral Workflow	24
8.4	Connection & Real-Time Notification Workflow	24
8.5	Real-Time Messaging Workflow	25
9	Frontend Architecture	26
9.1	Directory Structure	26
9.2	State Synchronization & Context Providers	26
9.3	Routing & Navigation	27
9.4	Design System & Styling	27
10	Backend Architecture	29
10.1	Modular Components	29
10.2	Request-Response Execution Pipeline	29
10.3	Global Exception Handler	30
11	Database Design & ER Diagram	32
11.1	Database Table Definitions	32
11.1.1	Users Table ('users')	32
11.1.2	User Details Table ('user_details')	32
11.1.3	Education Details Table ('education_details')	33
11.1.4	User Experiences Table ('user_experiences')	33
11.1.5	Projects Table ('projects')	33
11.1.6	Posts Table ('posts')	34
11.1.7	Comments Table ('comments')	34
11.1.8	Connections Table ('connections')	34
11.1.9	Chat Messages Table ('chat_messages')	35
11.1.10	Notifications Table ('notifications')	35
11.2	Database ER Diagram	35
12	API Documentation	36
12.1	Authentication & Session Gateway	36
12.1.1	Send Registration OTP	36
12.1.2	Complete Registration	36
12.1.3	Log In (Get JWT)	37
12.2	Profile Management	37
12.2.1	Fetch Active User Profile	37
12.3	Connection & Networking Actions	38
12.3.1	Request Connection	38
12.3.2	Accept Connection	38
12.3.3	Fetch Connection Status	38
12.4	Job Board Operations	38
12.4.1	Post Job Opportunity	38
12.5	WebSocket Channels	39
12.5.1	STOMP Endpoint	39
12.5.2	Subscribed Channels	39

13 Authentication Mechanics	40
13.1 Password Hashing via BCrypt	40
13.2 JWT Signature & Payload Claims	40
13.3 Request Filtering: 'JwtFilter'	40
13.4 WebSocket Handshake Authentication Interceptor	41
14 Security Architecture & Hardening	42
14.1 Implemented Security Controls	42
14.2 Future Security Recommendations	42
15 Deployment & Orchestration	44
15.1 Dockerization (Multi-Stage Build)	44
15.2 Kubernetes Orchestration	44
15.2.1 Deployment Controller ('deployment.yaml')	44
15.2.2 Service Mapping ('service.yaml')	45
15.2.3 Ingress Router Configuration ('ingress.yaml')	45
15.3 Jenkins CI/CD Pipeline	45
16 Testing & Automated Cleanup	46
16.1 Existing Test Suite	46
16.2 Developer Testing Shortcuts	46
16.3 Automated Test Data Cleanup Scheduler	46
16.4 Recommended Testing Strategy	47
17 Performance Tuning	48
17.1 Implemented Performance Configurations	48
17.2 Performance Tuning Recommendations	48
18 Scalability Blueprint	49
18.1 Horizontal Scaling of Backend Pods	49
18.2 Redis Message Broker for WebSocket Sync	49
18.3 Database Scaling: Read-Write Separation	50
18.4 Content Delivery Network (CDN) Integration	50
19 Troubleshooting & Operational Runbook	51
19.1 Database Connectivity Failures	51
19.2 CORS Policy Violations	51
19.3 WebSocket Handshake dropped	52
19.4 Amazon S3 Media Upload Failures	52
20 System Limitations	53
20.1 In-Memory OTP Cache Storage	53
20.2 Monolithic Database Constraints	53
20.3 Single-Node WebSocket Broker	53
20.4 Email Notification Gaps	53
20.5 Session Management Security	54
21 Feature & Architectural Roadmap	55
21.1 Short-Term Milestones (1–3 Months)	55
21.2 Medium-Term Milestones (3–6 Months)	55
21.3 Long-Term Milestones (6–12 Months)	55

22 Developer Contribution Guide	56
22.1 Branching Strategy & Workflow	56
22.2 Coding Standards & Formatting	56
22.2.1 Backend Java Code Standards	56
22.2.2 Frontend React Code Standards	56
22.3 Commit Message Conventions	57
22.4 Pre-PR Checklist	57
23 Frequently Asked Questions	58
23.1 Identity and Authentication	58
23.1.1 Why is registration limited to '@rgukt.ac.in' email addresses?	58
23.1.2 Can I sign up if I do not have a university email address?	58
23.1.3 How long are OTP codes valid for verification?	58
23.2 Real-Time Messaging	58
23.2.1 Can I message any member in the alumni directory?	58
23.2.2 Does the chat service support offline messaging?	58
23.3 Technical Architecture	58
23.3.1 How are profile pictures and media attachments stored?	58
23.3.2 Does the platform support horizontal scaling?	59
23.3.3 Why do my tests with email prefixes starting with 'test' not send emails?	59
24 Conclusion	60
24.1 Document Summary	60
24.2 Platform Outlook	60

List of Figures

3.1	RGUKT Connect User Journey and Platform Navigation Flow	13
6.1	RGUKT Connect Decoupled Architecture Blueprint	19
10.1	Backend Request-Response Pipeline and Middleware Filtering	30
11.1	RGUKT Connect Relational Database ER Schema Diagram	35
18.1	Distributed WebSocket Architecture using a Redis Pub/Sub Backplane	49

List of Tables

5.1	RGUKT Connect Role-Based Permission Matrix	16
7.1	RGUKT Connect Layered Technology Stack	21
9.1	Frontend Route Mappings and Access Rules	27

Chapter 1

Introduction

Rajiv Gandhi University of Knowledge Technologies (RGUKT) is a premier state university in India, established with the mission of providing high-quality technical education to rural youth. Operating across multiple campuses (including Nuzvid, RK Valley, Srikakulam, and Ongole), RGUKT has produced thousands of engineering graduates who are now employed in major global technology companies, research centers, and public service organizations.

Despite this large, talented, and highly dispersed alumni pool, there has historically been a significant digital gap between current students on campus and alumni in the industry. Communication has remained fragmented across unofficial channels, such as local WhatsApp groups, Facebook pages, and Telegram channels. These unofficial channels fail to provide a structured environment for professional networking, persistent knowledge sharing, or direct, secure job referrals.

1.1 Project Motivation

The motivation for building **RGUKT Connect** stems from three primary requirements:

1. **Institutional Trust Validation:** LinkedIn and other global professional networks do not verify academic records in a trustless manner. Anyone can claim an affiliation with RGUKT, which introduces a potential verification gap. There is an urgent need for a network limited strictly to verified RGUKT members.
2. **Direct Alumni Referral Channels:** Current recruitment processes are heavily congested. Standard online applications often fall into resume blackholes. Direct referrals from active employees inside companies offer the highest conversion rates for students. A platform that directly links students to alumni for referrals dramatically improves placement rates.
3. **Persistent Mentorship:** Students require structured feedback on resumes, projects, and interview preparation. Connecting them with alumni in matching engineering roles provides direct, high-value industry mentorship.

1.2 System Objectives

The primary objectives of the RGUKT Connect platform are to:

- Establish a secure platform restricted to verified students and alumni by verifying their university-issued email domain ('@rgukt.ac.in') and 7-character registration ID.
- Provide an interactive community feed where users can share professional news, code snippets, and post media.

- Implement a searchable directory to locate alumni by branch, batch, company, and role.
- Build a real-time messaging gateway to allow instant communication between connected users without sharing personal phone numbers or email addresses.
- Maintain a dedicated job board for verified alumni to post active job opportunities and internship openings.

1.3 Document Scope

This document serves as the complete technical and product blueprint for RGUKT Connect. It provides a comprehensive analysis of the system architecture, product characteristics, technical stack, database designs, authentication mechanisms, deployment methodologies, and operational workflows. It is intended for software engineers, deployment specialists, recruiters, and prospective collaborators seeking a deep architectural understanding of the platform.

Chapter 2

Problem Statement

In the professional networking landscape, students and graduates of Rajiv Gandhi University of Knowledge Technologies (RGUKT) face unique structural hurdles. To understand the need for a specialized university network, we must analyze the existing communication deficits and operational challenges.

2.1 Analysis of Current Communication Deficits

Currently, the RGUKT community relies on general-purpose social networks (such as LinkedIn, Facebook, and Twitter) or private messaging channels (such as WhatsApp and Telegram) to manage alumni-student communication. These general-purpose channels exhibit major limitations:

- **Fragmentation:** Unofficial messaging groups are restricted by size limits (e.g., WhatsApp group limits) and split by batches or branches. This splits the network into isolated clusters, preventing unified outreach.
- **High Signal-to-Noise Ratio:** General-purpose groups are often saturated with non-professional discussions, meme sharing, and administrative announcements, making it difficult for users to isolate high-value career discussions or referral posts.
- **Searchability Issues:** Messaging applications do not structure profile data. A student cannot run a query to find a senior who graduated in 2021 with a CSE degree and currently works as a cloud engineer at a specific target company.

2.2 The Verification Gap and Trust Deficit

A critical vulnerability in public networking websites is the absence of verification controls for institutional associations:

- **Affiliation Spoofing:** Anyone can create a public profile on LinkedIn and assert that they are a student or graduate of RGUKT. This lack of validation introduces a security vulnerability, opening the door for phishing and social engineering.
- **Spamming Alumni:** Because public networks are open, alumni are often flooded with generic connection requests and cold messages from strangers outside their university circle. This causes network fatigue, causing alumni to ignore legitimate student inquiries.

2.3 The Job Referral Bottleneck

Getting job interviews through standard resume portal applications has become increasingly difficult due to automated Applicant Tracking System (ATS) filtering. Direct referrals from employees bypass these filters and guarantee manual review. However, the path to a referral is blocked:

- **Lack of Direct Context:** Cold emailing or cold messaging alumni on LinkedIn without a shared context is often ignored.
- **Friction in Posting:** Alumni who want to help their juniors by referring them for active openings in their companies have no central university dashboard to post these listings. They must post on personal feeds, where reach is limited.

2.4 The Proposed Solution: RGUKT Connect

RGUKT Connect resolves these challenges by establishing a closed-loop, verified network restricted strictly to the RGUKT ecosystem. By verifying accounts through university email domains ('@rgukt.ac.in') and university-issued student registration IDs, the platform guarantees that every user is authentic. It integrates a structured directory, direct job and referral postings, a social feed, and real-time private messaging in a single, secure space.

Chapter 3

Product Overview

RGUKT Connect is designed as a secure, specialized, and real-time professional ecosystem. This chapter details the core product definition, its target user groups, boundaries, and the general user journey.

3.1 Target User Roles

The platform is designed around three main user groups within the university community:

1. **Students (Current Undergraduates):** Users seeking career guidance, mentorship, project reviews, and direct job or internship referrals. They use the platform to search for alumni, build professional profiles, participate in discussions, and apply for listings.
2. **Alumni (Graduated Professionals):** Users who are currently employed in the industry or pursuing advanced degrees. They act as mentors, write posts sharing industry trends, publish job openings from their employers, and offer direct referrals.
3. **Administrators (Platform Managers):** Users responsible for managing the platform. They can delete posts, manage jobs, check system health, and oversee user activity.

3.2 Platform Boundaries

To preserve security and institutional focus, the platform maintains strict operational boundaries:

- **Closed Registration:** Only users with emails ending in '@rgukt.ac.in' and 7-character registration numbers can sign up. External registrations are rejected.
- **Verification Loop:** System interaction is blocked until the registration email is verified using a 6-digit OTP delivered via SMTP.
- **Connection-Based Interaction:** To prevent spam, private messaging is restricted. A student can only initiate a direct chat with an alumnus or other student once they send a connection request and the recipient accepts it.

3.3 User Journey & Flow

The user journey is divided into authentication, discovery, and engagement. When a user first opens the web application, the root route checks the LocalStorage session token. If the token is missing or expired, the user is redirected to the public Landing page, where they can choose to

log in or register. Once authenticated, they enter the main platform containing the Home Feed, Network Directory, Jobs Board, Chat client, and Profile dashboard.

Figure 3.1 shows the complete user journey flowchart.

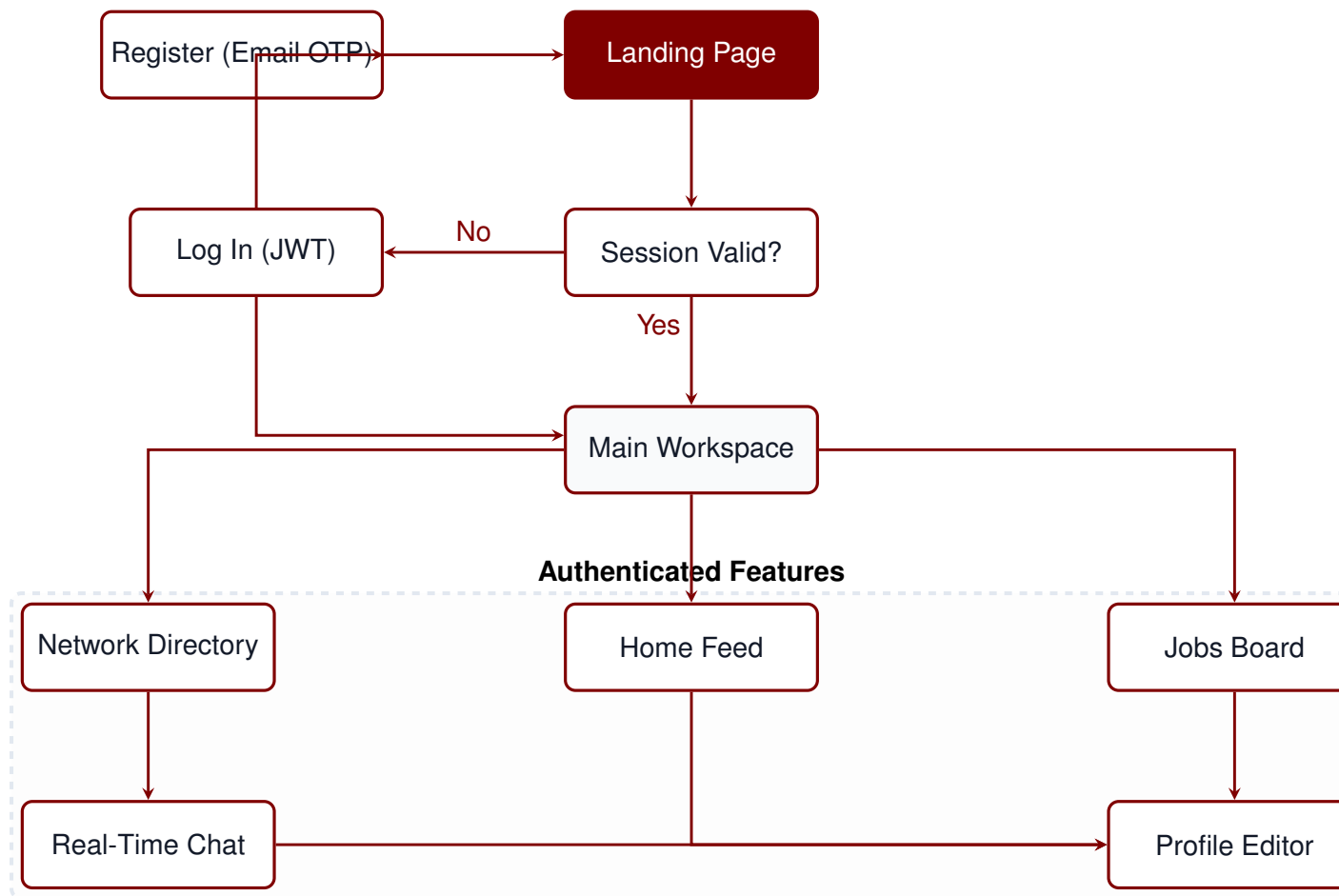


Figure 3.1: RGUKT Connect User Journey and Platform Navigation Flow

Chapter 4

Features

This chapter details the functional capabilities of the RGUKT Connect platform, including validation logic, storage configurations, and network constraints.

4.1 Identity Verification & OTP Lifecycle

Account security begins during registration and password recovery.

- **Registration Verification:** Users enter their name, university email, and student ID. The system validates that the email domain matches '@rgukt.ac.in' and the ID is exactly 7 characters. It generates a 6-digit OTP, stores it in an in-memory 'ConcurrentHashMap' with a 5-minute expiry, and sends it via SMTP.
- **Password Recovery:** The recovery process uses a similar workflow. The system verifies the user exists, generates an OTP, and requires its submission alongside a new password.
- **Testing Shortcut:** To streamline automated testing and development, emails starting with the prefix 'test' bypass SMTP sending. The system hardcodes the OTP to '123456'.

4.2 Alumni Directory & Connection Management

Discovering other members and establishing contact uses a directory and a connection workflow:

- **Directory Search:** Users can search the global registry, which lists name, batch, branch, profile picture, and current employment role. The directory automatically excludes the current user.
- **Relationship Matrix:** Connections use a state machine with the states:
 - NOT_CONNECTED: No relationship exists. Users see a "Connect" option.
 - PENDING_SENT: The current user sent a request. They see a "Pending" option.
 - PENDING_RECEIVED: The current user received a request. They see "Accept" and "Reject" options.
 - ACCEPTED: The connection is established. Users can send private messages.
- **Action Handling:** Rejecting or cancelling a request deletes the database row. Accepting updates the status to ACCEPTED and triggers a real-time notification to the sender.

4.3 Real-Time STOMP Messaging

Private messaging uses WebSockets to support real-time delivery:

- **STOMP Protocol:** The client connects using a STOMP client over WebSockets ('ws://<host>/ws-chat/websocket'). The connection handshake includes a JWT token in the 'Authorization' header, which is validated by a custom WebSocket channel interceptor.
- **Topic Subscriptions:** Users subscribe to '/user/queue/messages' for incoming messages. Sent messages are posted to '/api/chat/send' and saved to PostgreSQL.
- **Delivery Indicators:** The UI displays unread counts. If a user is not currently viewing the active chat, the app increments the unread count and plays a notification sound.

4.4 Opportunities Board & Job Referrals

To streamline employment opportunities, the platform provides a dedicated Job Board:

- **Role Restrictions:** Only users with the role 'ALUMNI' or 'ADMIN' can post jobs or mark posts as 'referral'. If a student attempts to post, the system returns a validation error.
- **Attributes:** Posts contain company name, role title, location, salary, application URL, employment type, expiration date, and a referral availability marker.

4.5 Community Feed & Media Integration

The central feed is an interactive dashboard where users share information:

- **Post Types:** Supports standard text posts, code snippets (with syntax highlighting), and media attachments.
- **Media Uploads:** Images and videos are uploaded to AWS S3. S3 keys follow the format:

$$\text{Key} = \text{posts}/\{idNumber\}-\{timestamp\}\{extension\} \quad (4.1)$$

- **Interactions:** Users can like posts and add nested comments. Like states are updated dynamically, and deleting a post cascades to delete all comments.

4.6 Profile Customization

Profiles display academic and professional backgrounds:

- **Data Collections:** Users can add, edit, and delete multiple education history blocks, professional experience listings, and personal portfolio projects.
- **Avatar Uploads:** Custom avatar uploads are saved to AWS S3 using the key:

$$\text{Key} = \text{profiles}/\{idNumber\}-\text{avatar}\{extension\} \quad (4.2)$$

When uploading a new picture, the system deletes the old image from S3.

Chapter 5

User Roles and Permissions

To maintain platform integrity and security, RGUKT Connect enforces Role-Based Access Control (RBAC). The application classifies users into three primary groups: Student, Alumni, and Admin.

5.1 Role Profiles

- **Student (STUDENT):** Current undergraduates at RGUKT. They can search the directory, send connection requests, post on the community feed, comment, like posts, search job postings, and chat with accepted connections. Students cannot post job opportunities or mark feed posts as career referrals.
- **Alumni (ALUMNI):** Verified graduates of RGUKT. They have all student privileges, plus authorization to post jobs on the Job Board and create referral posts.
- **Admin (ADMIN):** Platform managers. Admins have access to all alumni privileges and administrative overrides, including deleting any user's post, comment, or job listing.

5.2 Permission Matrix

Table 5.1 summarizes the authorization rules across the system.

Table 5.1: RGUKT Connect Role-Based Permission Matrix

System Action / Capability	Student	Alumni	Admin
Register	Authenticated	Authenticated	Authenticated
Log In (JWT Retrieval)	Yes	Yes	Yes
Modify Own Profile	Yes	Yes	Yes
Upload Profile Avatar (S3)	Yes	Yes	Yes
Query Alumni Directory	Yes	Yes	Yes
Send/Accept Connections	Yes	Yes	Yes
Publish Community Feed Post	Yes	Yes	Yes
Like and Comment on Posts	Yes	Yes	Yes
Delete Own Posts / Comments	Yes	Yes	Yes
Post Job Referral Listings	No	Yes	Yes
Publish Job Postings	No	Yes	Yes
Delete Any Post / Comment	No	No	Yes
Delete Any Job Listing	No	No	Yes
Bypass Email Delivery in Dev (Test OTP)	Yes	Yes	Yes

5.3 Security Policy Enforcements

Authorization is verified at both the frontend and backend levels:

1. **Client-Side Filtering:** The frontend reads the 'userSession' role string and conditionally hides actions like "Post Job" or "Post Referral" for students.
2. **Server-Side Verification:** If a client attempts to bypass frontend controls and calls a restricted endpoint (such as 'POST /api/jobs' or 'POST /api/posts' with type 'referral'), the backend service layer validates the user's role and rejects the request if they are not an alumnus or admin.

Chapter 6

System Architecture

RGUKT Connect uses a decoupled client-server architecture. This separation isolates the presentation layer from the business logic, enabling independent scaling and updates.

6.1 System Architecture Overview

The system architecture consists of three main tiers:

1. **Client Presentation Tier (Single-Page Application):** A React-based web application. It communicates with the backend via RESTful HTTP APIs and persistent WebSocket channels.
2. **API Ingress & Gateway Tier (Reverse Proxy):** An Ingress Controller (Nginx) that routes requests based on URL path prefixes:
 - `/api/` routes are sent to the Spring Boot REST API.
 - `/ws-chat/` connections are upgraded and sent to the STOMP WebSocket broker.
3. **Core Application Server (Spring Boot):** A Java-based backend application. It handles request filtering, security checks, STOMP broker routing, mail queue processing, and Hibernate transaction management.
4. **Storage & Persistence Tier:** Divided into two parts: PostgreSQL for structured relational data, and Amazon S3 for media files.

Figure 6.1 illustrates the system components and their relationships.

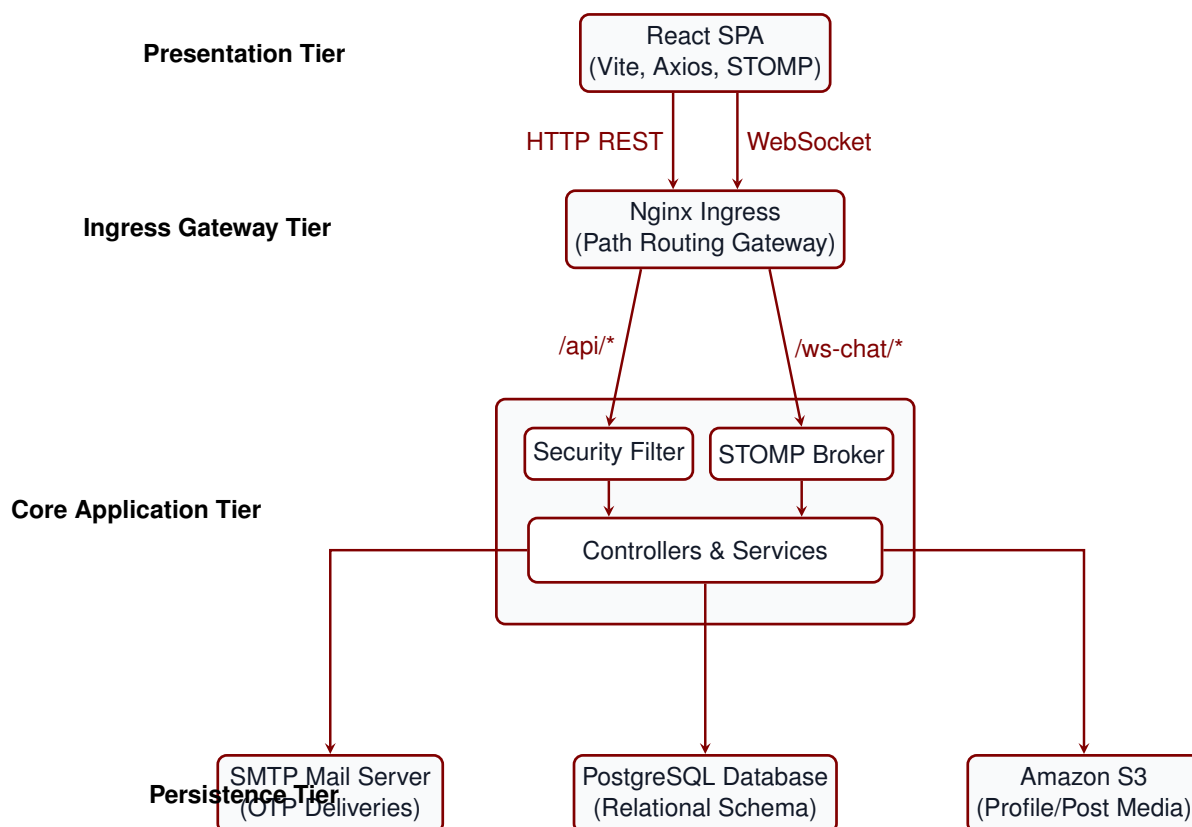


Figure 6.1: RGUKT Connect Decoupled Architecture Blueprint

6.2 Component Communication Protocols

The application components communicate using three primary protocols:

1. **HTTP/HTTPS REST (JSON):** Used for all standard request-response operations, including user registration, session logins, profile editing, post submissions, and job postings.
2. **STOMP over WebSocket:** Used for duplex real-time communications. Rather than using raw WebSockets, the platform runs STOMP on top, providing sub-protocols for frame routing.
3. **JDBC (Java Database Connectivity):** Used by the Spring Data JPA layer via a HikariCP connection pool to query and write to the PostgreSQL database.
4. **AWS SDK (HTTP REST):** The backend uses the AWS SDK to communicate with Amazon S3. Profile picture uploads generate public S3 URLs that are stored back in PostgreSQL.

Chapter 7

Technology Stack

The selection of the technology stack for RGUKT Connect was driven by considerations of security, developer productivity, execution performance, and deployment automation. This chapter outlines the software languages, frameworks, and storage systems used.

7.1 Tech Stack Matrix

Table 7.1 details each layer of the application.

Table 7.1: RGUKT Connect Layered Technology Stack

Layer	Technology / Tool	Version	Primary Purpose / Rationale
Frontend Core	React	19.2.5	Component-based UI rendering with state synchronization.
Frontend Build	Vite	8.0.10	Rapid hot module reloading and optimized ES module bundling.
Styling System	Tailwind CSS	4.2.4	Utility-first CSS theme compilation.
API Client	Axios	1.17.0	Promise-based HTTP client supporting request interception and custom headers.
WS Client	@stomp/stompjs	7.3.0	WebSockets STOMP protocol abstraction client.
Backend Engine	Spring Boot	3.4.x / 4.x	Enterprise-grade application runner.
Language	Java JDK	21	Modern features and performance improvements.
ORM Layer	Hibernate / Spring JPA	Default	Automatic entity mapping and transaction management.
DB Driver	PostgreSQL JDBC	42.x	Database connectivity driver.
Connection Pool	HikariCP	Default	High-speed JDBC connection pool.
Database Engine	PostgreSQL	15 / 16	Relational database.
Binary Storage	Amazon S3	SDK v1/v2	Secure object storage for media attachments.
Mail Server	JavaMail / SMTP	Default	Relational connection for OTP deliveries.
CI/CD Server	Jenkins	Latest	Automation server managing build pipelines.
Orchestration	Kubernetes	1.28+	Orchestrator running replicated pods.

7.2 Technical Rationale for Primary Choices

- **Java 21 & Spring Boot:** JDK 21 introduces virtual threads and enhanced pattern matching, improving application performance. Spring Boot provides pre-built security configurations, WebSocket support, and JPA integrations out of the box, reducing boilerplate code.
- **PostgreSQL:** Chosen over NoSQL options because the core data model (users, connections, comments, likes) is highly relational. PostgreSQL supports transaction management and ACID properties, which are critical for preserving connection states and notification structures.
- **React 19 & Vite:** React 19 improves concurrent rendering and state management. Vite

replaces Webpack to compile code faster, accelerating the development cycle.

- **Amazon S3 for Binaries:** Storing image files directly inside a database degrades query performance. By offloading media files to S3 and saving only their generated public URLs in the database, the database remains compact.

Chapter 8

Application Workflows

This chapter details the operational workflows of RGUKT Connect, outlining the steps, endpoints, and data transformations for key processes.

8.1 Registration & OTP Verification Workflow

Establishing a new user account requires dual verification: university email validation and OTP challenge.

1. **Identity Submission:** The user enters their name, ID number, and email.

2. **Backend Validation:** The frontend sends a request to:

```
1 POST /api/auth/register/send-otp?email={email}&idNumber={
2 idNumber}
```

The backend verifies the email domain is '@rgukt.ac.in', the ID is 7 characters, and that neither the email nor the ID is already registered.

3. **OTP Generation:** The backend generates a 6-digit OTP, saves it in an in-memory map ('registrationOtpStorage') with a 5-minute expiry, and sends it via SMTP.

4. **Bypass Check:** If the email starts with 'test', the system skips SMTP and sets the OTP to '123456'.

5. **Submission:** The user submits their registration details, password, and the OTP to:

```
1 POST /api/auth/register
2
```

6. **Commit Phase:** The backend validates the OTP. If valid, it hashes the password with BCrypt, saves a new 'User' record, creates a default 'UserDetails' entry (setting default branch to 'CSE' and phone to '9876543210'), and deletes the OTP from the in-memory map.

8.2 Authentication & JWT Lifecycle

Logging in establishes a stateless session:

1. **Credentials Submission:** The user submits their email and password to:

```
1 POST /api/auth/login
2
```

2. **Credential Verification:** The backend verifies the email domain. It retrieves the user, validates the BCrypt password hash, and returns an error if authentication fails.
3. **Token Issuance:** The system generates a JWT access token containing the user's email, ID number, and role, signed with the backend secret.
4. **Client Cache:** The backend returns the JWT token and basic profile metadata. The React client saves this data as a JSON string in LocalStorage under the key 'userSession'.
5. **Authorized Requests:** The client includes the token in the 'Authorization' header ('Bearer <token>') for all subsequent API requests.

8.3 Job Posting & Referral Workflow

Posting a job opportunity requires specific role privileges:

1. **Form Submission:** The user fills out the job details form.
2. **API Request:** The client sends the payload with the authorization token to:

```
1 POST /api/jobs
2
```

3. **Role Validation:** The backend extracts the user's role from the JWT. If the role is not 'ALUMNI' or 'ADMIN', the backend rejects the request with an HTTP 403 status code.
4. **DB Write:** If authorized, the backend creates a 'Job' record linked to the posting user.

8.4 Connection & Real-Time Notification Workflow

Establishing a connection updates the network graph and triggers notifications:

1. **Connection Request:** User A clicks "Connect" on User B's profile. The client sends a request to:

```
1 POST /api/connections/request/{userId}
2
```

2. **Database Entry:** The backend creates a 'Connection' record with a status of 'PENDING'.
3. **Notification Trigger:** The backend creates a 'Notification' entry with the type CONNECTION_REQUEST and the ID of the connection request.
4. **Real-Time Push:** The backend pushes the notification over WebSockets to the recipient:

```
1 /user/{recipientId}/queue/notifications
2
```

5. **Acceptance:** User B clicks "Accept". The client sends a request to:

```
1 PUT /api/connections/accept/{requestId}
2
```

The backend updates the connection status to 'ACCEPTED' and pushes a CONNECTION_ACCEPTED notification back to User A.

8.5 Real-Time Messaging Workflow

Private messaging uses WebSockets to support real-time delivery:

1. **STOMP Handshake:** The client establishes a WebSocket connection:

```
1 ws://<host>/ws-chat/websocket
2
```

A custom channel interceptor validates the JWT token in the 'Authorization' header.

2. **Subscription:** The client subscribes to:

```
1 /user/queue/messages
2
```

3. **Message Dispatch:** When sending a message, the client posts the JSON payload to '/api/chat/send' via an HTTP POST request.
4. **Persistence:** The backend saves the message to the database and checks if User B is currently online.
5. **WebSocket Delivery:** The system pushes the message to User B:

```
1 /user/{receiverId}/queue/messages
2
```

Chapter 9

Frontend Architecture

The frontend of RGUKT Connect is built as a single-page application (SPA) using React 19 and compiled with Vite. This chapter details the directory layout, state management architecture, navigation system, and theme configuration.

9.1 Directory Structure

The frontend source code is organized into folders to separate components, pages, and state context:

- **'src/main.jsx'**: Entry point of the application. Configures Axios defaults (e.g., setting the backend base URL via `import.meta.env.VITE_API_URL || 'http://localhost:8000'`).
- **'src/App.jsx'**: Core router component. Configures the route paths, wraps the application inside context providers, and verifies session states.
- **'src/context/'**: Contains React Context files that handle global states:
 - **'NotificationContext'**: Manages global STOMP connections, unread notification counts, and real-time alerts.
 - **'UserContext'**: Fetches and caches the logged-in user's profile.
 - **'HomeContext'**: Coordinates the community post feed, like actions, and comment additions.
 - **'JobsContext'**: Manages the active job boards, filtering, and job creations.
 - **'NetworkContext'**: Coordinates member search directories and connection invitations.
 - **'MessagesContext'**: Handles private chat histories and real-time message streams.
 - **'PromptContext'**: Coordinates modal warnings and confirmations.
- **'src/pages/'**: Contains page views, including Landing, Login, Register, ForgotPassword, Home, Network, Jobs, Messages, and Profile.
- **'src/components/'**: Houses reusable UI components (e.g., Navbar, UserCard, PostCard, modals).

9.2 State Synchronization & Context Providers

To prevent prop-drilling, global states are managed using React Context. The root node of the component tree is wrapped in the following provider hierarchy:

```

1 <NotificationProvider session={session}>
2   <PromptProvider>
3     <UserProvider session={session}>
4       <HomeProvider session={session} onLogout={handleLogout}>
5         <JobsProvider session={session} onLogout={handleLogout}>
6           <NetworkProvider session={session} onLogout={handleLogout}>
7             <MessagesProvider session={session} onLogout={handleLogout}>
8               <BrowserRouter> ... </BrowserRouter>
9             </MessagesProvider>
10          </NetworkProvider>
11        </JobsProvider>
12      </HomeProvider>
13    </UserProvider>
14  </PromptProvider>
15</NotificationProvider>

```

This setup ensures that whenever the authenticated ‘session’ transitions (e.g., on login or logout), the dependent contexts clean up their internal cache and reset their states.

9.3 Routing & Navigation

Routes are split into public pages and protected workspaces using a custom ‘ProtectedRoute’ wrapper. The route mappings are summarized in Table 9.1.

Table 9.1: Frontend Route Mappings and Access Rules

Route Path	Component	Access Level	Redirect Rule (If Unauthorized)
‘/’	‘Landing’	Public	None
‘/login’	‘Login’	Public (Anonymous)	Redirect to ‘/home’ if already authenticated
‘/register’	‘Register’	Public (Anonymous)	Redirect to ‘/home’ if already authenticated
‘/forgot-password’	‘ForgotPassword’	Public (Anonymous)	Redirect to ‘/home’ if already authenticated
‘/home’	‘Home’	Protected (Session)	Redirect to ‘/login’ if token missing
‘/network’	‘Network’	Protected (Session)	Redirect to ‘/login’ if token missing
‘/jobs’	‘Jobs’	Protected (Session)	Redirect to ‘/login’ if token missing
‘/messages’	‘Messages’	Protected (Session)	Redirect to ‘/login’ if token missing
‘/profile’	‘Profile’	Protected (Session)	Redirect to ‘/login’ if token missing
‘*’	(Fallback)	Redirect to ‘/’	N/A

9.4 Design System & Styling

Styling uses Tailwind CSS v4.0. The layout theme is customized in ‘src/index.css’ inside the ‘@theme’ block. Core variables include:

- **Brand Primary Color:** ‘—color-rgukt-maroon’ set to ‘#800000’ (burgundy).
- **Brand Accent Color:** ‘—color-rgukt-gold’ set to ‘#ffd700’ (yellow gold).
- **Background Neutral:** ‘—color-rgukt-slate’ set to ‘#f9fafb’ (off-white).
- **Font Stack:** Google Fonts **Inter** for body content, **Outfit** for headers, and **Source Code Pro** for code blocks.

Hover states and micro-animations use Tailwind utility modifiers (‘transition-all’, ‘hover:scale-[1.02]’, ‘active:scale-[0.98]’) to provide immediate visual feedback.

Chapter 10

Backend Architecture

The backend of RGUKT Connect is built on Spring Boot, running on Java 21. This chapter details the server architecture, directory layout, request-response execution pipeline, and exception handling mechanism.

10.1 Modular Components

The backend source code is organized into packages to separate concerns:

- **'com.uday.rguktconnect.controller'**: REST Controllers. They expose endpoints, handle request mapping, and validate parameters.
- **'com.uday.rguktconnect.service'**: Interface and implementation classes. They execute business logic, run database transaction boundaries, enforce validation constraints, and interface with external systems (such as AWS S3 and Java Mail).
- **'com.uday.rguktconnect.repository'**: Spring Data JPA Repositories. They define queries and database interaction boundaries.
- **'com.uday.rguktconnect.entity'**: Hibernate entities. They map Java objects to PostgreSQL tables.
- **'com.uday.rguktconnect.dto'**: Data Transfer Objects. They decoupled database schemas from API schemas.
- **'com.uday.rguktconnect.security'**: Custom filter chains, password encryptions, JWT utilities, and Web Security policies.
- **'com.uday.rguktconnect.exception'**: Global exception handlers that catch exceptions and translate them into standard JSON responses.

10.2 Request-Response Execution Pipeline

Every API request sent to the backend goes through a structured execution pipeline:

1. **Filter Chain (JWT Extraction)**: The request first hits the 'JwtFilter' class. It extracts the 'Authorization' header, validates the signature, extracts the claims (email, idNumber, role), and registers the authenticated user in the 'SecurityContext'.
2. **Security Configuration**: The request is verified against the security policy in 'Security-Config'. If unauthorized or calling a protected resource without a session, the request is rejected with an HTTP 401/403 status code.

3. **REST Controller Dispatch:** The request is routed to the corresponding controller method. The controller maps incoming JSON inputs to DTO objects.
4. **Service Execution:** The controller invokes the service layer. The service layer executes business logic (e.g., checking if a user has permission to delete a resource) and updates data inside transaction boundaries.
5. **Database Persistence:** The service invokes JPA repositories to execute database actions.

Figure 10.1 illustrates this pipeline.

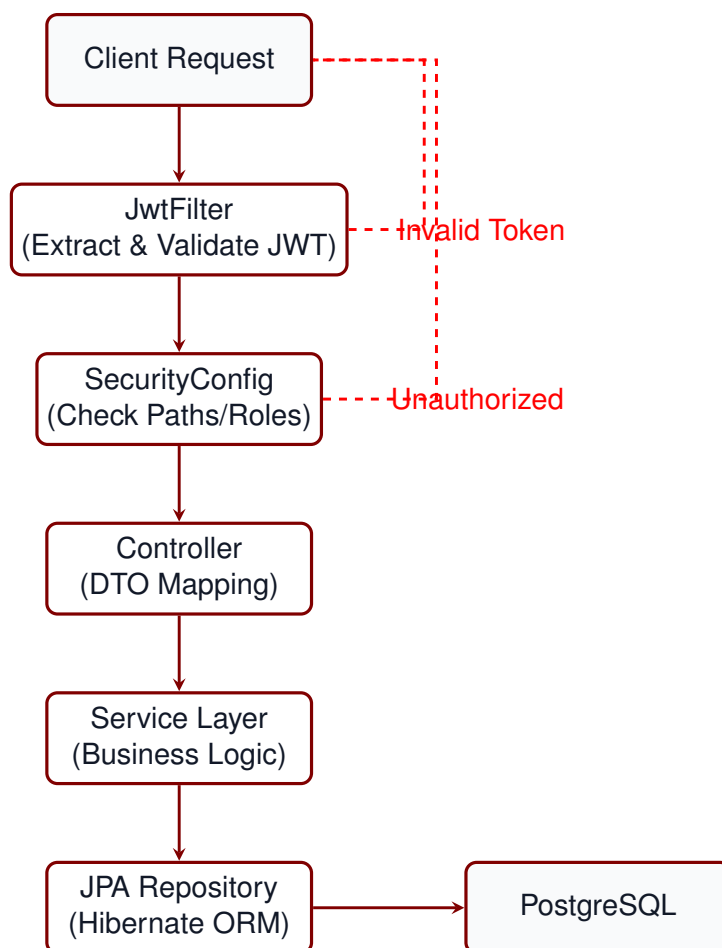


Figure 10.1: Backend Request-Response Pipeline and Middleware Filtering

10.3 Global Exception Handler

To prevent server trace leakage and provide clean API responses, all uncaught exceptions are caught by 'GlobalExceptionHandler':

```
1 @RestControllerAdvice
2 public class GlobalExceptionHandler {
3     @ExceptionHandler(RuntimeException.class)
4     public ResponseEntity<?> handleRuntimeException(RuntimeException ex) {
5         return ResponseEntity
6             .status(HttpStatus.BAD_REQUEST)
7             .body(Map.of("error", ex.getMessage()));
8     }
9 }
```

```
9 }
```

This advice intercepts runtime failures (such as validation failures or permission errors) and returns a JSON payload (`"error": "<exception message>"`) alongside a standard HTTP 400 Bad Request status code.

Chapter 11

Database Design & ER Diagram

RGUKT Connect uses a PostgreSQL database. The schema is mapped using Hibernate ORM with the 'update' DDL strategy. This chapter details the database entities, tables, relationships, and includes a detailed Entity-Relationship (ER) diagram.

11.1 Database Table Definitions

The relational database consists of ten primary tables.

11.1.1 Users Table ('users')

Stores the core authentication and identity records.

- **'id':** 'BIGINT' (Primary Key, Auto-increment).
- **'id_number':** 'VARCHAR(7)' (Unique, e.g., B211449).
- **'name':** 'VARCHAR(255)' (Full name).
- **'university_email':** 'VARCHAR(255)' (Unique, ending in '@rgukt.ac.in').
- **'password':** 'VARCHAR(255)' (BCrypt hash).
- **'role':** 'VARCHAR(50)' (e.g., STUDENT, ALUMNI, ADMIN).
- **'created_at':** 'TIMESTAMP' (Auto-generated).

11.1.2 User Details Table ('user_details')

Stores profile extensions. Linked to 'users' via a One-to-One relationship.

- **'id':** 'BIGINT' (Primary Key).
- **'user_id':** 'BIGINT' (Foreign Key pointing to 'users.id', Unique).
- **'mobile_number':** 'VARCHAR(15)' (Default: '9876543210').
- **'personal_email':** 'VARCHAR(255)' (Optional).
- **'branch':** 'VARCHAR(50)' (Academic branch, e.g., CSE, ECE).
- **'batch':** 'VARCHAR(10)' (e.g., 2021).
- **'profile_photo':** 'VARCHAR(500)' (S3 URL).

- **'description':** 'TEXT' (Professional bio).
- **'github_url' / 'linkedin_url':** 'VARCHAR(500)'.
- **'mentored_students_count':** 'INT' (Default: 0).

11.1.3 Education Details Table ('education_details')

Tracks the academic history of users. Linked via a Many-to-One relationship to 'users'.

- **'id':** 'BIGINT' (Primary Key).
- **'user_id':** 'BIGINT' (Foreign Key pointing to 'users.id').
- **'institution_name':** 'VARCHAR(255)'.
- **'degree':** 'VARCHAR(100)'.
- **'field_of_study':** 'VARCHAR(255)'.
- **'start_year' / 'end_year':** 'INT'.
- **'grade':** 'VARCHAR(50)'.

11.1.4 User Experiences Table ('user_experiences')

Tracks the professional history of users. Linked via a Many-to-One relationship to 'users'.

- **'id':** 'BIGINT' (Primary Key).
- **'user_id':** 'BIGINT' (Foreign Key pointing to 'users.id').
- **'title' / 'company_name' / 'location':** 'VARCHAR(255)'.
- **'employment_type' / 'location_type':** 'VARCHAR(100)' (e.g., Remote, On-site).
- **'start_date' / 'end_date':** 'DATE'.
- **'is_current_role':** 'BOOLEAN'.
- **'description':** 'TEXT'.

11.1.5 Projects Table ('projects')

Tracks user projects. Linked via a Many-to-One relationship to 'users'.

- **'id':** 'BIGINT' (Primary Key).
- **'user_id':** 'BIGINT' (Foreign Key pointing to 'users.id').
- **'title':** 'VARCHAR(255)'.
- **'description':** 'TEXT'.
- **'project_url' / 'repo_url':** 'VARCHAR(500)'.

11.1.6 Posts Table ('posts')

Stores community feed items. Linked to 'users' as author.

- **'id'**: 'BIGINT' (Primary Key).
- **'author_id'**: 'BIGINT' (Foreign Key pointing to 'users.id').
- **'type'**: 'VARCHAR(50)' (e.g., text, referral, image).
- **'content'**: 'TEXT'.
- **'code_snippet'**: 'TEXT' (Optional, supports code shares).
- **'company' / 'role'**: 'VARCHAR(255)' (Optional, for referral type).
- **'media_url'**: 'VARCHAR(500)' (AWS S3 file attachment).
- **'created_at'**: 'TIMESTAMP'.

11.1.7 Comments Table ('comments')

Nested post comment structures. Linked to 'posts' and 'users' (as author).

- **'id'**: 'BIGINT' (Primary Key).
- **'post_id'**: 'BIGINT' (Foreign Key pointing to 'posts.id').
- **'author_id'**: 'BIGINT' (Foreign Key pointing to 'users.id').
- **'parent_id'**: 'BIGINT' (Self-referential Foreign Key pointing to 'comments.id' for nested replies).
- **'content'**: 'TEXT'.
- **'created_at'**: 'TIMESTAMP'.

11.1.8 Connections Table ('connections')

Handles networking relationships. Includes a unique constraint on the pair '(sender_id, receiver_id)'.

- **'id'**: 'BIGINT' (Primary Key).
- **'sender_id'**: 'BIGINT' (Foreign Key pointing to 'users.id').
- **'receiver_id'**: 'BIGINT' (Foreign Key pointing to 'users.id').
- **'status'**: 'VARCHAR(50)' (e.g., PENDING, ACCEPTED).
- **'created_at' / 'updated_at'**: 'TIMESTAMP'.

11.1.9 Chat Messages Table ('chat_messages')

Persistent history for chat messages.

- 'id': 'BIGINT' (Primary Key).
- 'sender_id': 'BIGINT' (Foreign Key pointing to 'users.id').
- 'receiver_id': 'BIGINT' (Foreign Key pointing to 'users.id').
- 'content': 'TEXT' (Encrypted or raw text storage).
- 'is_read': 'BOOLEAN' (Default: false).
- 'timestamp': 'TIMESTAMP'.

11.1.10 Notifications Table ('notifications')

Tracks alerts.

- 'id': 'BIGINT' (Primary Key).
- 'recipient_id': 'BIGINT' (Foreign Key pointing to 'users.id').
- 'sender_id': 'BIGINT' (Foreign Key pointing to 'users.id').
- 'type': 'VARCHAR(100)' (e.g., CONNECTION_REQUEST, CONNECTION_ACCEPTED).
- 'related_id': 'BIGINT' (Polymorphic resource ID).
- 'is_read': 'BOOLEAN'.
- 'created_at': 'TIMESTAMP'.

11.2 Database ER Diagram

Figure 11.1 shows the database schema relationships.

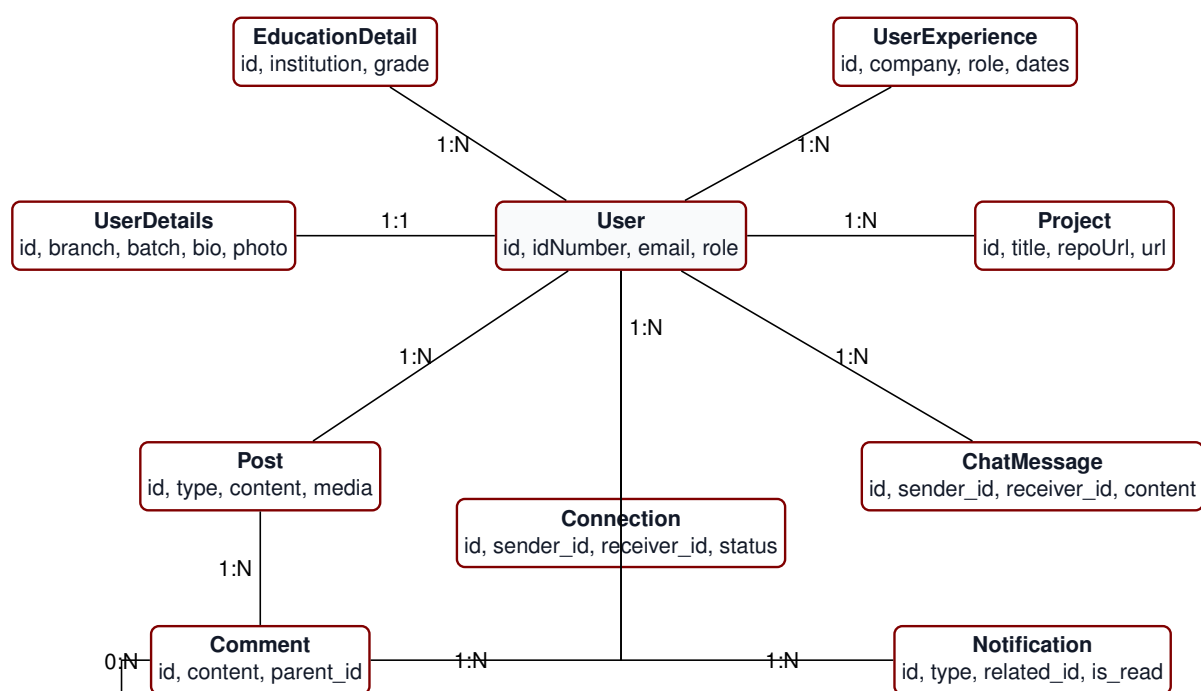


Figure 11.1: RGUKT Connect Relational Database ER Schema Diagram

Chapter 12

API Documentation

RGUKT Connect exposes REST endpoints for transactional workflows and WebSocket endpoints for real-time messaging. This chapter defines the API pathways, parameters, and request/response JSON schemas.

12.1 Authentication & Session Gateway

12.1.1 Send Registration OTP

Sends a verification OTP to the user's university email address.

- **Path:** 'POST /api/auth/register/send-otp'
- **Parameters (Query):**
 - 'email' (string, required): e.g., 'B211449@rgukt.ac.in'
 - 'idNumber' (string, required): e.g., 'B211449'
- **Response:** '200 OK' on success.

12.1.2 Complete Registration

Registers a new user account after validating the email OTP.

- **Path:** 'POST /api/auth/register'
- **Request Body (JSON):**

```
1  {  
2    "idNumber": "B211449",  
3    "name": "Jane Doe",  
4    "universityEmail": "B211449@rgukt.ac.in",  
5    "password": "securepassword123",  
6    "role": "STUDENT",  
7    "otp": "123456"  
8  }  
9
```

- **Response Body (JSON - 200 OK):**

```
1  {  
2    "id": 15,  
3    "idNumber": "B211449",
```

```
4     "name": "Jane Doe",
5     "universityEmail": "B211449@rgukt.ac.in",
6     "role": "STUDENT",
7     "createdAt": "2026-08-18T12:00:00Z"
8   }
9
```

12.1.3 Log In (Get JWT)

Authenticates credentials and returns a JWT access token.

- **Path:** 'POST /api/auth/login'
- **Request Body (JSON):**

```
1   {
2     "universityEmail": "B211449@rgukt.ac.in",
3     "password": "securepassword123"
4   }
5
```

- **Response Body (JSON - 200 OK):**

```
1   {
2     "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3     "user": {
4       "id": 15,
5       "idNumber": "B211449",
6       "name": "Jane Doe",
7       "universityEmail": "B211449@rgukt.ac.in",
8       "role": "STUDENT",
9       "createdAt": "2026-08-18T12:00:00Z"
10    }
11  }
12
```

12.2 Profile Management

12.2.1 Fetch Active User Profile

Retrieves the profile details of the authenticated user.

- **Path:** 'GET /api/users/profile'
- **Headers:** 'Authorization: Bearer <token>'
- **Response Body (JSON - 200 OK):**

```
1   {
2     "id": 15,
3     "idNumber": "B211449",
4     "name": "Jane Doe",
5     "universityEmail": "B211449@rgukt.ac.in",
6     "mobileNumber": "9876543210",
7     "personalEmail": "jane.doe@gmail.com",
8     "branch": "CSE",
9     "batch": "2021",

```

```
10     "profilePhoto": "https://rgukt-connect-bucket.s3.amazonaws.com/profiles/B211449-avatar.png",
11     "description": "Software engineer intern",
12     "githubUrl": "https://github.com/janedoe",
13     "linkedinUrl": "https://linkedin.com/in/janedoe",
14     "mentoredStudentsCount": 0,
15     "projects": [],
16     "experiences": [],
17     "education": []
18 }
19
```

12.3 Connection & Networking Actions

12.3.1 Request Connection

Sends a pending connection invitation to another user.

- **Path:** 'POST /api/connections/request/receiverId'
- **Headers:** 'Authorization: Bearer <token>'
- **Response:** '200 OK' (creates request and triggers WS notification).

12.3.2 Accept Connection

Accepts a pending connection request.

- **Path:** 'PUT /api/connections/accept/requestId'
- **Headers:** 'Authorization: Bearer <token>'
- **Response:** '200 OK' (updates state and triggers WS notification).

12.3.3 Fetch Connection Status

Checks the connection state between the active user and a target user.

- **Path:** 'GET /api/connections/status/targetUserId'
- **Headers:** 'Authorization: Bearer <token>'
- **Response Body (JSON - 200 OK):**

```
1  {
2    "status": "PENDING_RECEIVED",
3    "connectionId": 48
4  }
5
```

12.4 Job Board Operations

12.4.1 Post Job Opportunity

Publishes a job or internship opportunity. Requires the role 'ALUMNI' or 'ADMIN'.

- **Path:** 'POST /api/jobs'

- **Headers:** 'Authorization: Bearer <token>'

- **Request Body (JSON):**

```
1  {
2    "company": "Google",
3    "role": "Systems Engineer",
4    "location": "Hyderabad, India",
5    "salary": "25 LPA",
6    "applyUrl": "https://careers.google.com/jobs/...",
7    "type": "Full-time",
8    "expiresAt": "2026-10-31",
9    "referralAvailable": true,
10   "category": "Software Engineering"
11 }
12
```

- **Response Body (JSON - 200 OK):**

```
1  {
2    "id": 8,
3    "company": "Google",
4    "role": "Systems Engineer",
5    "location": "Hyderabad, India",
6    "salary": "25 LPA",
7    "applyUrl": "https://careers.google.com/jobs/...",
8    "type": "Full-time",
9    "postedBy": "Alumni John",
10   "postedById": 3,
11   "referralAvailable": true,
12   "category": "Software Engineering",
13   "createdAt": "2026-08-18T15:30:00Z",
14   "expiresAt": "2026-10-31"
15 }
16
```

12.5 WebSocket Channels

12.5.1 STOMP Endpoint

- **URL:** 'ws://<host>/ws-chat/websocket'
- **Handshake Headers:** 'Authorization: Bearer <token>'

12.5.2 Subscribed Channels

- **‘/user/queue/messages’:** Receives real-time private messages from connected users.
- **‘/user/queue/notifications’:** Receives real-time connection alerts.

Chapter 13

Authentication Mechanics

RGUKT Connect uses stateless token-based authentication. This chapter details password hashing, JSON Web Token (JWT) signatures, request-level verification filters, and WebSocket handshake authentication.

13.1 Password Hashing via BCrypt

User passwords are encrypted before storage. The system uses Spring Security's 'BCryptPasswordEncoder' with a default strength factor of 10. BCrypt incorporates a random salt for each hash, securing stored passwords against rainbow table attacks:

```
1 String securePassword = passwordEncoder.encode(requestDTO.getPassword()  
    );  
2 user.setPassword(securePassword);
```

During login, the system matches the raw password against the stored hash using:

```
1 passwordEncoder.matches(loginRequest.getPassword(), user.getPassword())
```

13.2 JWT Signature & Payload Claims

Upon successful authentication, the server generates a signed JWT access token.

- **Signature Algorithm:** HMAC-SHA256 ('HS256').
- **Signing Key:** Loaded from the 'JWT_SECRET' environment variable.
- **Payload Claims:** The JWT contains:
 - Subject ('sub'): University email address.
 - Issued At ('iat'): Timestamp of creation.
 - Expiration ('exp'): Timestamp when the token expires.
 - Custom Claims: User ID number and account Role.

13.3 Request Filtering: 'JwtFilter'

For HTTP requests, the custom 'JwtFilter' class (which extends 'OncePerRequestFilter') processes the security context:

1. It extracts the 'Authorization' header from the incoming request.

2. It checks if the header begins with the prefix 'Bearer '.
3. It parses the token using the secret key and extracts the email and user roles.
4. If the token is valid, it builds a 'UsernamePasswordAuthenticationToken' and sets it in Spring Security's context:

```
1 SecurityContextHolder.getContext().setAuthentication(  
  authenticationToken);  
2
```

13.4 WebSocket Handshake Authentication Interceptor

Since WebSockets are upgrade connections that do not use standard HTTP filters after the initial handshake, securing STOMP frames requires a different approach.

The application registers a custom channel interceptor in 'WebSocketConfig' that intercepts incoming STOMP command headers during the connection handshake ('CONNECT' frame):

1. The client includes the JWT token in the 'Authorization' header inside the STOMP connection headers.
2. The 'configureClientInboundChannel' interceptor extracts the 'Authorization' header.
3. It parses and validates the token.
4. If valid, it extracts the user's details and registers a custom 'Principal' object linked to the user's unique Database ID (converted to a string).
5. This mapping allows the backend to route private messages to specific users using:

```
1 messagingTemplate.convertAndSendToUser(recipientId.toString(),  
  "/queue/messages", messageDto);  
2
```

Chapter 14

Security Architecture & Hardening

This chapter reviews the security measures implemented in RGUKT Connect and outlines recommendations for future hardening.

14.1 Implemented Security Controls

- **Identity Verification Restrictions:** Only users with '@rgukt.ac.in' emails can register. Student registration IDs are checked to ensure they contain exactly 7 characters, preventing external sign-ups.
- **In-Memory OTP Protection:** OTP codes are stored in-memory using 'ConcurrentHashMap' caches with an automatic 5-minute timeout. This prevents long-term storage of temporary codes.
- **Stateless Authorization:** System actions require a JWT token. This eliminates session hijacking risks on the server by removing all server-side session state.
- **CORS Configuration:** The backend enforces CORS policies using the 'CORS_ALLOWED_ORIGINS' environment variable. The filter checks the request origin and blocks requests from unauthorized domains.
- **S3 File Storage Isolation:** S3 uploads are saved using unique paths linked to user IDs and timestamps. The application does not accept custom file names, protecting the system against directory traversal attacks.
- **SQL Injection Prevention:** Database queries use Spring Data JPA's parameterized methods. The Hibernate engine compiles these queries into prepared statements, preventing SQL injection vulnerabilities.

14.2 Future Security Recommendations

1. **SSL/TLS Encryption:** Endpoints must use HTTPS. Production deployments should configure Nginx Ingress to terminate TLS certificates (using Let's Encrypt or similar tools) to encrypt data in transit.
2. **Rate Limiting on OTP Endpoints:** The API currently does not restrict how often a user can call 'send-otp'. Attackers could exploit this to flood the SMTP queue or spam users. Implementing a rate limiter (such as Bucket4j or Redis token bucket) would prevent abuse.

3. **JWT Blocklisting with Redis:** Because JWT tokens are stateless, they cannot be invalidated before their expiration date. Implementing a Redis-based blocklist would allow users to log out immediately or enable administrators to revoke compromised tokens.
4. **Secrets Management:** The production environments should load database passwords, S3 keys, and JWT secrets from a secure secrets provider (such as HashiCorp Vault or Kubernetes Secrets) instead of plain text environment files.
5. **Input Sanitization:** To protect the platform against Cross-Site Scripting (XSS) attacks in the community feed, the client and server should sanitize user input, scrubbing HTML tags and scripts from post bodies and code snippets.

Chapter 15

Deployment & Orchestration

This chapter details the deployment architecture of RGUKT Connect, covering containerization, Kubernetes orchestration, ingress routing, and the CI/CD pipeline.

15.1 Dockerization (Multi-Stage Build)

The backend uses a multi-stage Docker build to keep production images small and secure. The compiler environment (JDK) is separated from the runtime environment (JRE).

Listing 15.1 shows the backend 'Dockerfile'.

```
1 # Stage 1: Build stage
2 FROM maven:3.9.9-eclipse-temurin-21 AS build
3 WORKDIR /app
4 COPY pom.xml .
5 RUN mvn dependency:go-offline
6 COPY . .
7 RUN mvn clean package -DskipTests
8
9 # Stage 2: Runtime stage
10 FROM eclipse-temurin:21-jre
11 WORKDIR /app
12 COPY --from=build /app/target/*.jar app.jar
13 EXPOSE 8000
14 ENTRYPOINT ["java", "-jar", "app.jar", "--server.port=8000"]
```

Listing 15.1: Backend Multi-Stage Dockerfile

15.2 Kubernetes Orchestration

The platform is designed to run in a Kubernetes cluster, providing horizontal scaling and self-healing.

15.2.1 Deployment Controller ('deployment.yaml')

Manages replicas and deployment updates.

- **Replicas:** 2 pods running the image 'udayangari/rgukt-connect-backend:v2'.
- **Rolling Update Strategy:** Configured with 'maxSurge: 1' and 'maxUnavailable: 0'. This guarantees zero-downtime updates by spinning up a new container before stopping the old one.

- **Probes:**
 - **Readiness Probe:** Calls HTTP `/api/auth/health` on port 8000 after a 90-second delay to check when the pod is ready to receive traffic.
 - **Liveness Probe:** Calls HTTP `/api/auth/health` on port 8000 after a 120-second delay to monitor container health.
- **Resource Allocations:**
 - CPU: Request 250m, Limit 500m.
 - Memory: Request 512Mi, Limit 1Gi.

15.2.2 Service Mapping ('service.yaml')

The backend pods are exposed inside the cluster using a NodePort Service:

- **Service Name:** 'backend-service'
- **Type:** 'NodePort'
- **Ports:** Internal port 8000 mapped to NodePort 32001. This exposes the service on port 32001 across all cluster nodes.

15.2.3 Ingress Router Configuration ('ingress.yaml')

An Nginx Ingress Controller handles external routing:

- **API Routing:** Routes `/api(/|$)(.*)` requests to `backend-service:8000`.
- **WebSocket Upgrade Routing:** Routes WebSocket connections on path `/ws-chat(/|$)(.*)` directly to the backend pods to support real-time communication.

15.3 Jenkins CI/CD Pipeline

The build pipeline is automated using a Jenkins declarative pipeline ('Jenkinsfile'). The deployment process consists of ten stages:

1. **Checkout:** Pulls the latest source code from the Git repository.
2. **Verify Environment File:** Verifies the presence of the environment file `/opt/rgukt-connect/.env`.
3. **Docker Hub Login:** Authenticates with Docker Hub using Jenkins credentials ('dockerhub-creds').
4. **Build Docker Image:** Compiles and packages the application inside the 'backend' folder, tag-naming the resulting image.
5. **Push Image:** Pushes the image to Docker Hub.
6. **Stop Old Container:** Stops and removes the active container on the host machine.
7. **Remove Old Image:** Deletes the stale container image.
8. **Pull Latest Image:** Fetches the new build.
9. **Run Backend:** Starts the container in the background, loading environment variables from `/opt/rgukt-connect/.env` and mapping port 8000.
10. **Health Check:** Runs a polling loop that sends requests to `http://localhost:8000/health` up to 30 times (every 2 seconds) to confirm the server started successfully.

Chapter 16

Testing & Automated Cleanup

This chapter details the testing strategies and automated test data cleanup mechanisms implemented in RGUKT Connect.

16.1 Existing Test Suite

The application contains the standard Spring Boot context initialization test:

```
1 @SpringBootTest
2 class RguktConnectApplicationTests {
3     @Test
4     void contextLoads() {
5     }
6 }
```

This test executes during the build phase ('mvn clean package') to verify that the application context, Spring beans, database configurations, and environment dependencies load correctly.

16.2 Developer Testing Shortcuts

To support manual testing and local development, the platform provides two developer features in the authentication layer:

1. **Bypass Email Verification:** For emails starting with the prefix 'test' (e.g., 'teststudent@rgukt.ac.in' or 'testalumni@rgukt.ac.in'), the system skips the SMTP email sending process.
2. **Hardcoded OTP Key:** For these test accounts, the OTP is set to '123456'. This allows developers and automated scripts to register and reset passwords without access to a real email inbox.

16.3 Automated Test Data Cleanup Scheduler

To prevent database bloating from test accounts, the system runs an automated cleanup scheduler. A background thread ('CleanupScheduler') runs every 5 minutes ('fixedRate = 300000' milliseconds) using Spring's '@Scheduled' annotation:

- **Cutoff Period:** The scheduler defines a cutoff time of exactly 2 days before the current execution time:

```
1     LocalDateTime cutoff = LocalDateTime.now().minusDays(2);
2
```

- **Post Deletions:** The scheduler identifies posts created by accounts with the 'test' prefix that are older than the 2-day cutoff. It deletes all nested comment replies, top-level comments, and then deletes the post entity.
- **Connection Deletions:** The scheduler identifies pending connection requests involving 'test' accounts that are older than 2 days. It deletes associated connection request notifications and removes the connection entity.

16.4 Recommended Testing Strategy

To expand test coverage, the following testing plan is recommended:

- **Unit Testing (JUnit 5 + Mockito):** Target the service layer ('UserService', 'PostService', 'ConnectionService') to verify business rules (such as validating email domains and role permissions) using mocked database states.
- **Controller Integration Testing (MockMvc):** Write integration tests using '@WebMvcTest' to verify that API controllers return the correct HTTP status codes and JSON schemas.
- **WebSocket STOMP Testing:** Build a test client to verify that messaging frames are routed correctly over the STOMP protocol.
- **Frontend End-to-End (E2E) Testing:** Use Playwright or Cypress to automate user flows (such as registration, connections, and real-time chat) in a browser.

Chapter 17

Performance Tuning

This chapter details the performance configurations implemented in RGUKT Connect and recommends tuning strategies for future scaling.

17.1 Implemented Performance Configurations

- **JPA Lazy Fetching:** Entity mappings use lazy fetching (`FetchType.LAZY`) for collections (such as 'likedBy' in 'Post' and 'likedBy' in 'Comment'). This prevents the system from loading related records until they are requested.
- **HikariCP Connection Pool:** Database connections are managed by a HikariCP pool. HikariCP reduces connection latency by keeping a pool of active database connections open, avoiding the overhead of establishing a new TCP connection for every request.
- **Binary Offloading to S3:** Profile pictures and post attachments are offloaded directly to Amazon S3. The database only stores the resulting URL strings, keeping database size small and reducing query latency.
- **Index-Optimized Relationships:** Core tables use auto-generated indexes on primary and foreign keys, speeding up join operations.

17.2 Performance Tuning Recommendations

1. **Resolve N+1 Query Issues:** When loading the community feed, the application maps each 'Post' entity to a 'PostResponseDTO'. This maps user details, comment counts, and like states, which can trigger additional queries for each post in the list (the N+1 query problem). Using entity graphs or fetch joins ('JOIN FETCH') in custom JPQL queries would allow the system to load this data in a single query.
2. **Query Pagination:** Endpoints like 'GET /api/posts' and 'GET /api/jobs' load all records in a single request. As the platform grows, loading all records will degrade response times. The endpoints should use Spring Data's 'Pageable' interface to paginate results (e.g., loading 20 posts at a time).
3. **API Caching with Redis:** Read-heavy endpoints that change infrequently (such as the alumni directory search and job board listings) should be cached using Redis. This reduces database load and speeds up response times.
4. **Frontend Asset Optimizations:** The React client should implement code splitting using 'React.lazy' and 'Suspense' to load route components on demand, reducing the size of the initial JavaScript bundle.

Chapter 18

Scalability Blueprint

This chapter details the architectural design and roadmap for scaling RGUKT Connect to support high user traffic and concurrent messaging.

18.1 Horizontal Scaling of Backend Pods

The Spring Boot backend is packaged as a stateless Docker container. This allows the application to be scaled horizontally by running multiple replicas in the Kubernetes cluster. Traffic is distributed across these replicas using a Kubernetes Service.

However, scaling the stateless API layer introduces challenges for features that manage state, such as WebSockets and in-memory caches.

18.2 Redis Message Broker for WebSocket Sync

Currently, the WebSocket STOMP broker is run in-memory within each backend process. If a client connects to Pod A, and another connects to Pod B, they cannot exchange messages directly because their sessions are isolated on different servers.

To support multiple backend pods, the application must use a shared message broker (such as RabbitMQ or a Redis Pub/Sub backplane) instead of the in-memory broker.

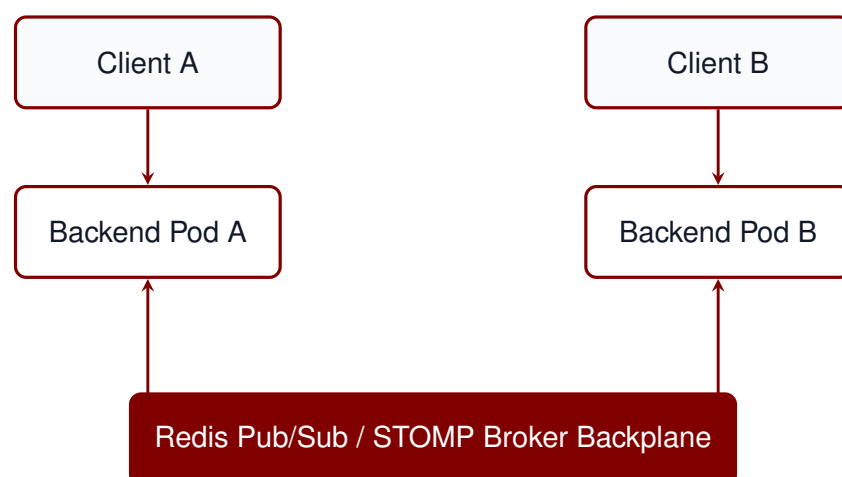


Figure 18.1: Distributed WebSocket Architecture using a Redis Pub/Sub Backplane

As shown in Figure 18.1, when Pod A receives a message, it publishes it to the Redis channel. Pod B, which is subscribed to the Redis channel, receives the message and forwards it to Client B, enabling cross-pod messaging.

18.3 Database Scaling: Read-Write Separation

Because user activity in social platforms is read-heavy (e.g., browsing feeds, searching directories, checking jobs), the database can become a bottleneck. The following database architecture is recommended for scaling:

- **Write Master Database:** A single database instance that processes all write operations (e.g., new posts, connections, chat logs).
- **Read Replicas:** Multiple read-only databases synced asynchronously with the master. The application routes read operations (such as directory queries) to these replicas, distributing the database load.

18.4 Content Delivery Network (CDN) Integration

To reduce load on the origin server, static assets (such as profile images and post attachments uploaded to Amazon S3) should be served through a CDN (such as Amazon CloudFront or Cloudflare). The CDN caches these assets at edge locations close to users, reducing latency and bandwidth costs.

Chapter 19

Troubleshooting & Operational Runbook

This chapter details the troubleshooting steps for common operational issues encountered during development, deployment, or system executions.

19.1 Database Connectivity Failures

- **Symptom:** The Spring Boot application fails to boot, throwing a 'HikariPool-1 - Connection is not available' error.
- **Cause:** The application cannot establish a socket connection to PostgreSQL, typically due to incorrect credentials, an inactive database process, or an unmapped database URL.

- **Resolution:**

1. Verify that the PostgreSQL service is active:

```
1      systemctl status postgresql
2
```

2. Verify that the environment variables ('DB_URL', 'DB_USERNAME', 'DB_PASSWORD') are loaded by the process.
3. For local development, check that the database 'rgukt_connect' exists in PostgreSQL.

19.2 CORS Policy Violations

- **Symptom:** API requests fail in the browser with the error: 'Access to XMLHttpRequest has been blocked by CORS policy'.
- **Cause:** The client application's origin does not match the origins allowed by the backend.
- **Resolution:**
 1. Check the 'CORS_ALLOWED_ORIGINS' environment variable in the backend configuration.
 2. Ensure it contains the frontend origin (e.g., 'http://localhost:5173' or 'http://localhost:3000') without trailing slashes.

19.3 WebSocket Handshake dropped

- **Symptom:** The frontend STOMP client fails to connect, throwing a protocol error or returning an HTTP 401 Unauthorized status code.
- **Cause:** The connection handshake header is missing the authorization token, the JWT token is expired, or the ingress gateway is not configured to upgrade HTTP connections to WebSockets.
- **Resolution:**

1. Check that the client includes the token in the STOMP connection headers:

```
1      connectHeaders: { Authorization: "Bearer " + token }
2
```

2. If deploying in Kubernetes, verify that the Nginx Ingress Controller configuration permits connection upgrades by checking these headers:

```
1      nginx.ingress.kubernetes.io/websocket-services: "
2      backend-service"
```

19.4 Amazon S3 Media Upload Failures

- **Symptom:** Uploading profile pictures or post attachments fails, and the backend logs throw an 'AmazonS3Exception: Access Denied' error.
- **Cause:** The backend has missing or incorrect AWS credentials, or the target S3 bucket does not exist or lacks read/write permissions.
- **Resolution:**

1. Check the AWS environment variables: 'AWS_ACCESS_KEY_ID', 'AWS_SECRET_ACCESS_KEY', 'AWS_REGION', and 'AWS_BUCKET_NAME'.
2. Ensure the IAM user associated with the credentials has 's3:PutObject', 's3:GetObject', and 's3:DeleteObject' permissions for the target bucket.

Chapter 20

System Limitations

This chapter details the current technical limitations of the RGUKT Connect platform.

20.1 In-Memory OTP Cache Storage

- **Limitation:** Registration and password recovery OTPs are stored in-memory using 'ConcurrentHashMap' instances.
- **Impact:** This design makes the authentication layer stateful. If the backend server restarts, crashes, or scales horizontally to multiple pods, users will experience validation errors when submitting OTPs, as the validation cache is tied to the memory space of a single server instance.

20.2 Monolithic Database Constraints

- **Limitation:** All data (users, connections, posts, comments, notifications, chat logs) is stored in a single PostgreSQL instance.
- **Impact:** The database is a single point of failure (SPOF) and a potential bottleneck as query traffic scales.

20.3 Single-Node WebSocket Broker

- **Limitation:** The WebSocket STOMP service uses Spring's default 'SimpleBroker', which runs in-memory within the application process.
- **Impact:** As discussed in the scalability blueprint (Chapter 18), this setup prevents the system from running multiple backend replicas in production. Messages cannot be routed between users connected to different server instances.

20.4 Email Notification Gaps

- **Limitation:** The system sends email alerts (via SMTP) only for registration and password recovery OTPs.
- **Impact:** Important community events (such as receiving a new connection request, connection acceptances, or direct messages) only trigger in-app notifications. If a user is offline, they will not receive email notifications about these events, which can reduce active user engagement.

20.5 Session Management Security

- **Limitation:** The frontend stores JWT tokens in LocalStorage ('userSession').
- **Impact:** Storing sensitive session data in LocalStorage makes the application vulnerable to Cross-Site Scripting (XSS) attacks. If an attacker injects a malicious script, they can access the token and compromise the user session.

Chapter 21

Feature & Architectural Roadmap

This chapter details the feature additions and architectural improvements planned for future updates to the RGUKT Connect platform.

21.1 Short-Term Milestones (1–3 Months)

- **Distributed Cache Migration:** Replace the in-memory OTP ‘ConcurrentHashMap’ caches with a Redis instance. This will make the authentication layer stateless, allowing the backend to scale horizontally without session dropouts.
- **Email Notification Alerts:** Implement email notifications (via SMTP) for connection requests, connection acceptances, and private messages received while offline.
- **Basic Chat Search:** Add search functionality to the chat client, allowing users to query past discussions by keywords.

21.2 Medium-Term Milestones (3–6 Months)

- **WebSocket Backplane Integration:** Integrate a Redis Pub/Sub backplane with Spring’s STOMP broker. This enables cross-pod messaging, allowing the backend to scale horizontally across multiple instances in the Kubernetes cluster.
- **Mobile Application Development:** Build a mobile application using React Native or Flutter, reusing the existing REST and WebSocket APIs.
- **Resume Hosting:** Extend the profile dashboard to support uploading PDF resumes, saving them to AWS S3 and indexing them for recruiter searches.

21.3 Long-Term Milestones (6–12 Months)

- **Multi-Tenant Architecture:** Refactor the backend to support multi-tenancy. This will allow other educational institutions to spin up their own verified connection networks on the platform.
- **AI-Powered Recommendation Engine:** Implement a semantic matching engine that analyzes profile data, experience history, and skills to suggest connections, mentors, and job postings.
- **Secure Cookie Authentication:** Replace LocalStorage-based session tokens with HTTP-only, secure, SameSite cookies to protect the application against Cross-Site Scripting (XSS) token theft.

Chapter 22

Developer Contribution Guide

This chapter outlines the guidelines for developers contributing to the RGUKT Connect codebase, covering branching strategies, coding standards, and deployment tests.

22.1 Branching Strategy & Workflow

The repository uses a Git Flow branching model:

- **‘main’**: Production branch. Contains stable, tested releases. Direct commits to ‘main’ are restricted.
- **‘develop’**: Integration branch. All features and bug fixes are integrated here first.
- **‘feature/*’**: Feature development branches. Created from ‘develop’ and merged back via Pull Requests (PRs).
- **‘hotfix/*’**: Urgent production fixes. Created from ‘main’ and merged into both ‘main’ and ‘develop’.

22.2 Coding Standards & Formatting

Developers must follow consistent formatting standards across layers.

22.2.1 Backend Java Code Standards

- Use standard camelCase naming for variables and methods, and PascalCase for classes.
- Maintain a clean separation of concerns: Controller classes must not execute business logic directly, and service classes must not interact with database JDBC layers directly.
- Document public API endpoints using Swagger or SpringDoc annotations.

22.2.2 Frontend React Code Standards

- Write functional components using standard ES6 syntax and React Hooks.
- Manage local states with the ‘useState’ hook, and use custom Context hooks for shared global state.
- Enforce styling rules using Tailwind CSS utility classes, and extract complex layout blocks into reusable UI components.

22.3 Commit Message Conventions

Commits should use structured, semantic prefixes:

- **‘feat:’** A new functional capability (e.g., ‘feat: integrate S3 avatar upload’).
- **‘fix:’** A bug fix (e.g., ‘fix: resolve CORS header mismatch’).
- **‘docs:’** Documentation updates (e.g., ‘docs: add deployment instructions’).
- **‘refactor:’** Code refactoring that does not change functionality (e.g., ‘refactor: optimize JPA fetch join’).
- **‘test:’** Adding or modifying tests (e.g., ‘test: add user service unit tests’).

22.4 Pre-PR Checklist

Before opening a Pull Request, developers must complete the following steps:

1. Ensure the application compiles without errors:

```
1  # Backend check
2  mvn clean compile
3
4  # Frontend check
5  npm run build
6
```

2. Run all context and unit tests:

```
1  mvn test
2
```

3. Verify formatting and linting:

```
1  npm run lint
2
```

4. Test API endpoints using local REST clients (such as Postman or Bruno).

Chapter 23

Frequently Asked Questions

This chapter answers common technical and functional questions about the RGUKT Connect platform.

23.1 Identity and Authentication

23.1.1 Why is registration limited to '@rgukt.ac.in' email addresses?

Limiting registration to university-issued email domains prevents external users from signing up, protecting the platform's community focus and security.

23.1.2 Can I sign up if I do not have a university email address?

No. The platform requires a valid '@rgukt.ac.in' email to complete the registration and verification workflow.

23.1.3 How long are OTP codes valid for verification?

Both registration and password recovery OTPs are valid for exactly 5 minutes from the time they are generated.

23.2 Real-Time Messaging

23.2.1 Can I message any member in the alumni directory?

No. To prevent spam and unsolicited messages, users must first send a connection request. Private messaging is enabled only after the recipient accepts the connection request.

23.2.2 Does the chat service support offline messaging?

Yes. If a user is offline, messages are saved in the database. When the user logs back in, the client retrieves unread messages from the backend history endpoint.

23.3 Technical Architecture

23.3.1 How are profile pictures and media attachments stored?

Media files are uploaded to Amazon S3. The database only stores the resulting public S3 URL, keeping the database footprint small and improving query performance.

23.3.2 Does the platform support horizontal scaling?

The API gateway and Spring Boot backend are stateless and can scale horizontally in Kubernetes. However, the default WebSocket STOMP broker is stateful and runs in-memory, which requires a Redis Pub/Sub backplane for horizontal scaling.

23.3.3 Why do my tests with email prefixes starting with 'test' not send emails?

To simplify development and automated testing, the backend skips the SMTP email sending process for emails with the prefix 'test' and sets the OTP to '123456'.

Chapter 24

Conclusion

RGUKT Connect addresses the professional networking gap within the Rajiv Gandhi University of Knowledge Technologies (RGUKT) community. By building a secure, closed-loop network restricted to verified students and alumni, the platform provides a trusted space for peer-to-peer mentorship, real-time collaboration, and direct job referrals.

24.1 Document Summary

This technical documentation has detailed the system's architecture and operational design:

- **System Architecture:** The platform uses a decoupled split-architecture, separating the React Vite frontend from the Java Spring Boot backend.
- **Data Management:** Structured relational data is stored in a PostgreSQL database, while large binary files (such as profile pictures and post attachments) are offloaded to Amazon S3.
- **Real-Time Communication:** Instant messaging and notification deliveries use STOMP over WebSockets, secured with custom channel interceptors.
- **Automated Operations:** The system uses background schedulers to clean up test data and automates building and deployment using Jenkins and Kubernetes.

24.2 Platform Outlook

As the platform grows, transitioning to a stateless caching model with Redis, scaling WebSockets with a shared message broker, and implementing cookie-based session management will prepare RGUKT Connect to scale and support future university communities.

Bibliography

- [1] The Kubernetes Authors. Kubernetes orchestration for containerized applications. <https://kubernetes.io/>, 2026.
- [2] Facebook. React: A javascript library for building user interfaces. <https://react.dev/>, 2025.
- [3] STOMP Specification Group. Stomp: Simple text orientated messaging protocol. <https://stomp.github.io/>, 2012.
- [4] Michael Jones, John Bradley, and Nat Sakimura. Rfc 7519: Json web token (jwt), 2015.
- [5] The Jenkins Project. Jenkins automation server. <https://www.jenkins.io/>, 2026.
- [6] O. O. Regina and S. H. Leo. *PostgreSQL: Up and Running*. O'Reilly Media, 2017.
- [7] Amazon Web Services. Amazon simple storage service (s3) documentation. <https://aws.amazon.com/s3/>, 2026.
- [8] Craig Walls. *Spring Boot in Action*. Manning Publications, 2016.